TaskListTable UI設計書

メタデータ

- **コンポーネントID**: TaskListTable
- **コンポーネント名**: タスク一覧テーブル
- **ファイルパス**: `frontend/src/components/ExperimentWorker/TaskListTable.tsx`
- **利用権限**: 実験作業者（experiment_worker）
- **作成日**: 2026-02-20
- **最終更新日**: 2026-02-20

---

1. 概要

1.1 コンポーネントの目的

選択されたメソッドに紐づくタスクを一覧表示し、各タスクに対するアクション（指示確認、開始、問題確認、実験記録、強制終了）を提供するテーブルコンポーネントです。

1.2 主要機能

- タスク一覧の表示（テーブル形式）
- タスク検索・フィルタリング機能
- タスクステータスによる絞り込み
- 進捗率クリックでフロー図表示
- 各種アクションボタン（指示、開始、問題確認、記録、強制終了）
- ソート機能（タスク名、ステータス、進捗率）

1.3 使用場所

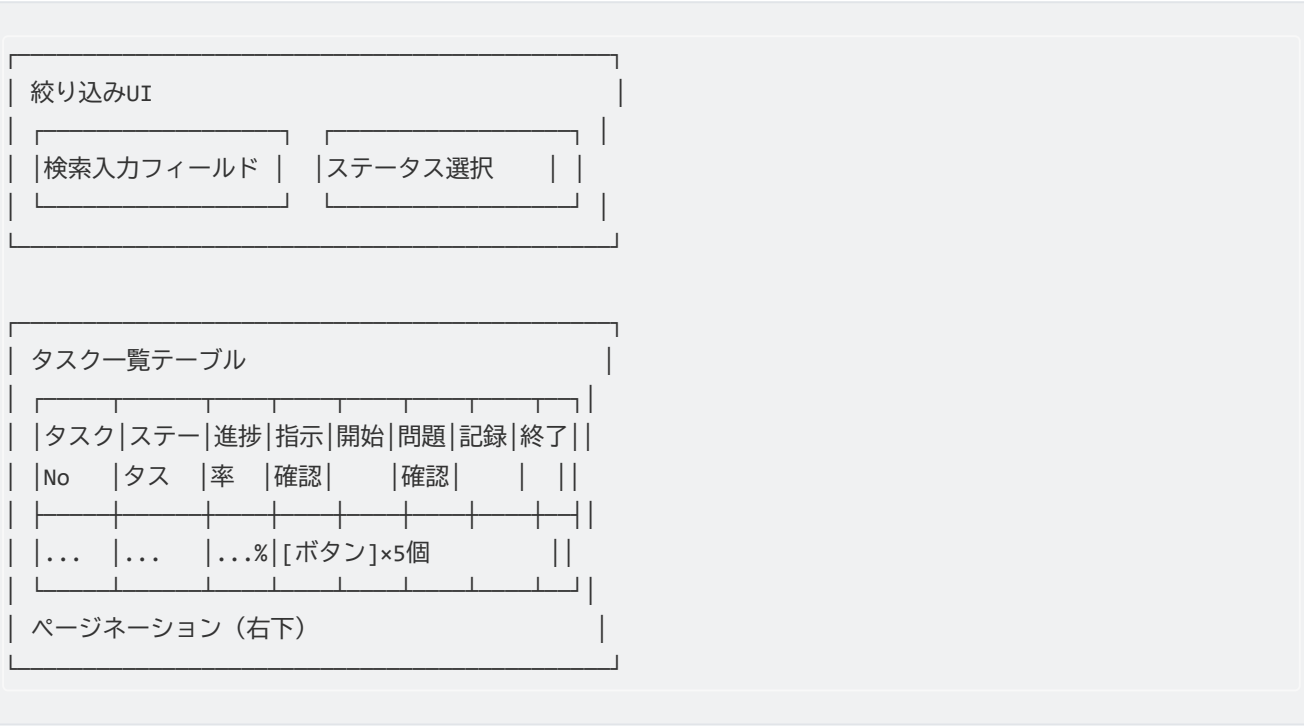
- ProgressCheckTable（進捗確認テーブル）内で使用

- ProgressCheckTab（進捗確認タブ）のメソッド別タブコンテンツ

---

2. コンポーネント構成

2.1 レイアウト構造



2.2 使用コンポーネント

Ant Designコンポーネント

- `Table` - メインテーブル
- `Input` - 検索フィールド
- `Select` - ステータス選択
- `Button` - アクションボタン
- `Tag` - ステータスバッジ
- `Typography.Text` - テキスト表示
- `Row`, `Col` - レイアウトグリッド

カスタムコンポーネント

- なし（純粋なプレゼンテーションコンポーネント）

---

3. Props

```
interface TaskListTableProps {
  tasks: Task[];                                    // タスク一覧
  selectedMethodId: string | null;                  // 選択されたメソッドID（現在未使用）
  projectStatus: ProjectStatus | null;              // プロジェクトステータス
  onTaskAction: (taskId: string, action: string) => void; // タスクアクションハンドラ
  onProgressRateClick: (task: Task) => void;        // 進捗率クリックハンドラ
  readOnly?: boolean;                               // trueのとき強制終了ボタンを無効化（オーバーレイ閲覧モード用）
}
```

Props詳細

Prop名	型	必須	説明
tasks	Task[]	✓	表示するタスク一覧（親コンポーネントでメソッドによるフィルタリング済み）
selectedMethodId	string | null	✓	選択されたメソッドID（現在は使用されていない、将来の拡張用）
projectStatus	ProjectStatus | null	✓	プロジェクトのステータス（ボタンの有効/無効判定に使用）
onTaskAction	function	✓	タスクアクションボタンクリック時のハンドラ
onProgressRateClick	function	✓	進捗率クリック時のハンドラ（フロー図表示）
readOnly	boolean	-	trueのとき強制終了ボタンを無効化。オーバーレイ閲覧モード用。デフォルト: false

---

4. State（内部状態）

4.1 絞り込み用State

```
// 検索テキスト
const [searchText, setSearchText] = useState<string>('');

// 選択されたステータス
const [selectedStatus, setSelectedStatus] = useState<TaskStatus | undefined>(undefined);
```

---

5. イベント

5.1 検索・フィルタリングイベント

イベント名	トリガー	処理内容
検索テキスト変更	Input onChange	searchTextを更新し、filteredTasksを再計算
ステータス選択変更	Select onChange	selectedStatusを更新し、filteredTasksを再計算

検索ロジック:

```
const filteredTasks = useMemo(() => {
  return tasks.filter(task => {
    // あいまい検索（タスクNo、ステータスラベル、進捗率）
    const matchesSearch = searchText === '' ||
      task.taskNo.toLowerCase().includes(searchText.toLowerCase()) ||
      getStatusLabel(task.status).includes(searchText) ||
      task.progressRate.toString().includes(searchText);

    // ステータスフィルタ
    const matchesStatus = !selectedStatus || task.status === selectedStatus;

    return matchesSearch && matchesStatus;
  });
}, [tasks, searchText, selectedStatus]);
```

5.2 進捗率クリックイベント

イベント名	トリガー	処理内容
handleProgressRateClick	進捗率テキストクリック	onProgressRateClick(task)を呼び出し、フロー図ダイアログを表示

実装:

```
<Text
  strong
  style={{
    fontSize: '14px',
    color: '#0093CF',
    cursor: 'pointer',
    textDecoration: 'underline'
  }}
  onClick={() => onProgressRateClick(record)}
  aria-label={`進捗率 ${rate}% - クリックしてフロー図を表示`}
>
  {rate}%
</Text>
```

5.3 アクションボタンイベント

ボタン名	アクション	処理内容	有効条件
指示	`instruction`	onTaskAction(taskId, 'instruction')を呼び出し	常に有効
開始	`start`	onTaskAction(taskId, 'start')を呼び出し。タスクステータス更新API(`PATCH /api/v1/progress/projects/:projectName/methods/:methodId/tasks/:taskNo/status`)でステータスを'in_progress'に更新。**楽観的ロック用に現在の `updatedAt` を `currentUpdatedAt` パラメータとして送信**。成功時はタスク一覧を再取得して最新の状態を反映。競合エラー（409 Conflict）発生時は最新のタスク一覧を再取得してユーザーに通知。	status === 'not_started' かつ isEditableStatus(projectStatus)
問題確認	`issueCheck`	onTaskAction(taskId, 'issueCheck')を呼び出し	常に有効

記録	`record`	onTaskAction(taskId, 'record')を呼び出し	常に有効
強制終了	`forceTerminate`	onTaskAction(taskId, 'forceTerminate')を呼び出し	status !== 'completed' かつ status !== 'forced_terminated' かつ isEditableStatus(projectStatus) かつ readOnly !== true

ボタンの有効/無効ロジック:

```
// 開始ボタン
const isProjectEditable = projectStatus ? isEditableStatus(projectStatus) : true;
const isDisabled = record.status !== 'not_started' || !isProjectEditable;

// 強制終了ボタン
const isDisabled =
  record.status === 'completed' ||
  record.status === 'forced_terminated' ||
  !isProjectEditable ||
  readOnly;  // readOnly=trueの場合も無効化（オーバーレイ閲覧モード用）
```

---

6. データフロー

6.1 Props → 内部State → 表示

```
親コンポーネント（ProgressCheckTable）
  ↓ props: tasks, projectStatus, onTaskAction, onProgressRateClick
TaskListTable
  ↓ state: searchText, selectedStatus
useMemo: filteredTasks（絞り込み）
  ↓
Table（Ant Design）
  ↓
各行のレンダリング
```

6.2 ユーザー操作のフロー

```
ユーザー操作
  ↓
1. 検索入力 → searchText更新 → filteredTasks再計算 → テーブル再描画
2. ステータス選択 → selectedStatus更新 → filteredTasks再計算 → テーブル再描画
3. 進捗率クリック → onProgressRateClick(task)実行 → 親でダイアログ表示
4. アクションボタンクリック → onTaskAction(taskId, action)実行 → 親で処理
```

---

7. テーブルカラム定義

7.1 カラム一覧

カラム名	dataIndex	width	align	sorter	説明
タスクNo	taskNo	95px	left	✓	タスクNoを表示
ステータス	status	95px	center	✓	タスクステータスをTagで表示
進捗率	progressRate	75px	center	✓	進捗率（クリック可能）
指示確認	-	85px	center	-	指示確認ボタン
開始	-	65px	center	-	タスク開始ボタン
問題確認（多言語対応）	-	65px	center	-	問題確認ボタン（タイトルは t('experimentWork.progressCheck.issueCheck')）
実験記録	-	85px	center	-	実験記録ボタン
強制終了	-	85px	center	-	強制終了ボタン

合計幅: 750px（scroll={{ x: 750 }}で横スクロール対応）

7.2 カラム詳細

7.2.1 タスクNo

```
{
  title: 'タスクNo',
  dataIndex: 'taskNo',
  key: 'taskNo',
  width: 95,
  ellipsis: true,
  sorter: (a, b) => a.taskNo.localeCompare(b.taskNo),
  render: (text: string) => (
    <Text style={{ fontSize: '14px' }}>{text}</Text>
  )
}
```

特徴:

- taskNoフィールドを表示
- ellipsis: trueで長いテキストを省略表示
- sorterでアルファベット順ソート可能
- フォントサイズ: 14px

7.2.2 ステータス

```
{
  title: 'ステータス',
  dataIndex: 'status',
  key: 'status',
  width: 95,
  align: 'center',
  sorter: (a, b) => a.status.localeCompare(b.status),
  render: (status: TaskStatus) => {
    const config = getStatusConfig(status);
    return (
      <Tag
        color={config.color}
        style={{ margin: 0 }}
        aria-label={`ステータス: ${config.label}`}
      >
        {config.label}
      </Tag>
    );
```

```
  }
}
```

ステータス表示:

TaskStatus値	表示ラベル	色
not_started	開始前	default（灰色）
in_progress	作業中	orange（オレンジ色）
completed	完了	green（緑色）
forced_terminated	強制終了	red（赤色）

7.2.3 進捗率

```
{
  title: '進捗率',
  dataIndex: 'progressRate',
  key: 'progressRate',
  width: 75,
  align: 'center',
  sorter: (a, b) => a.progressRate - b.progressRate,
  render: (rate: number, record: Task) => (
    <Text
      strong
      style={{
        fontSize: '14px',
        color: '#0093CF',
        cursor: 'pointer',
        textDecoration: 'underline'
      }}
      onClick={() => onProgressRateClick(record)}
      aria-label={`進捗率 ${rate}% - クリックしてフロー図を表示`}
    >
      {rate}%
    </Text>
  )
}
```

特徴:

- クリック可能（カーソル: pointer、下線付き）
- 色: #0093CF（青色）

- クリックでフロー図ダイアログを表示
- 数値でソート可能

7.2.4 アクションボタンカラム

共通仕様:

- size: "small"
- align: "center"
- onClick時にonTaskAction(taskId, action)を呼び出し

個別仕様:

1. **指示確認ボタン**
    - type: "default"
    - ラベル: "指示"
    - 常に有効
2. **開始ボタン**
    - type: "primary"
    - ラベル: "開始"
    - 有効条件: status === 'not_started' かつ isEditableStatus(projectStatus)
3. **問題確認ボタン**
    - type: "default"
    - ラベル: t('experimentWork.progressCheck.checkButton')
    - 常に有効
    - 多言語対応
4. **実験記録ボタン**
    - type: "default"
    - ラベル: "記録"
    - 常に有効
5. **強制終了ボタン**

- danger: true（赤色ボタン）
- ラベル: "終了"
- 有効条件: status !== 'completed' かつ status !== 'forced_terminated' かつ isEditableStatus(projectStatus)

---

8. データ構造

8.1 Task型

```
interface Task {
  taskId: string;              // タスクID（rowKeyとして使用）
  projectId?: string;          // プロジェクトID
  methodId?: string;           // メソッドID
  methodName: string;          // メソッド名
  taskNo: string;              // タスクNo
  taskName: string;            // タスク名
  sampleNumber: string;        // サンプル番号
  notes: string;               // 備考
  status: TaskStatus;          // ステータス
  assignedUserId?: string;     // 割り当てユーザーID
  assignedUserName?: string;   // 割り当てユーザー名
  createdAt?: string;          // 作成日時（ISO8601形式）
  updatedAt: string;           // 更新日時（ISO8601形式、楽観的ロック用に必須）
  progressRate: number;        // 進捗率（0-100）
  transitionStatuses?: TransitionStatus[]; // トランジションステータス一覧
  progressReport: string;      // 進捗報告の状態
  recordNumber: string;        // 実験記録番号
}
```

8.2 ProjectStatus型

```
type ProjectStatus =
  | 'planning'           // 計画中
  | 'waiting_task'       // タスク実行待ち
  | 'executing_task'     // タスク実行中
```

```
  | 'waiting_approval'   // 承認待ち
  | 'completed';         // 完了
```

8.3 TaskStatus型

```
type TaskStatus =
  | 'not_started'       // 開始前
  | 'in_progress'       // 作業中
  | 'completed'         // 完了
  | 'forced_terminated'; // 強制終了
```

---

9. ページネーション設定

9.1 ページネーション仕様

```
pagination={{
  pageSize: 20,                  // デフォルトページサイズ
  showSizeChanger: true,         // ページサイズ変更可能
  showQuickJumper: false,        // クイックジャンプ無効
  showTotal: (total) => `全${total}件`,
  pageSizeOptions: ['10', '20', '50', '100'],
  locale: {
    items_per_page: '件/ページ',
    jump_to: 'ページ',
    page: ''
  }
}}
```

特徴:

- デフォルト20件/ページ
- 10、20、50、100件から選択可能
- 日本語ローカライズ

---

10. エラーチェック

10.1 現在の実装

現在、エラーチェックは実装されていません。Propsで渡されたtasksが空配列の場合は、Ant DesignのTableコンポーネントが自動的に「タスクがありません」と表示します。

```
locale={{
  emptyText: 'タスクがありません'
}}
```

10.2 将来の実装

1. **データ取得エラー**
   - APIエラー時のエラーメッセージ表示
   - リトライ機能
2. **検索・フィルタエラー**
   - 不正な検索条件のバリデーション
   - フィルタ結果が0件の場合の表示
3. **アクション実行エラー**
   - ボタンクリック時のエラーハンドリング
   - 親コンポーネントへのエラー伝播

---

11. パフォーマンス考慮事項

11.1 useMemoの使用

```
// filteredTasksはuseMemoでメモ化
const filteredTasks = useMemo(() => {
  return tasks.filter(task => {
    // 絞り込みロジック
  });
}, [tasks, searchText, selectedStatus]);

// columnsもuseMemoでメモ化
const columns: ColumnsType<Task> = useMemo(() => {
  // カラム定義
}, [projectStatus, t, onTaskAction, onProgressRateClick]);
```

効果:

- tasksやsearchTextが変わらない限り、filteredTasksを再計算しない
- 不要な再レンダリングを防ぐ

11.2 大量データへの対応

- **ページネーション**: 20件/ページで分割表示
- **仮想スクロール**: 現在は未実装（将来の改善点）
- **横スクロール**: scroll={{ x: 750 }}で横幅が狭い画面でも表示可能

11.3 検索のデバウンス

現在は未実装ですが、将来的には検索入力にデバウンス処理を追加することで、入力中の過剰な再計算を防ぐことができます。

---

12. アクセシビリティ

12.1 ARIA属性

すべてのインタラクティブ要素にaria-labelを付与：

```
// 検索フィールド
aria-label="タスク一覧検索"
```

```
// ステータス選択
aria-label="タスクステータスで絞り込み"

// 進捗率
aria-label={`進捗率 ${rate}% - クリックしてフロー図を表示`}

// アクションボタン
aria-label="作業指示を確認"
aria-label="タスクを開始"
// 他のボタンも同様
```

12.2 キーボード操作

- テーブル行のフォーカス移動（Ant Designデフォルト）
- Tab/Shift+Tabでのボタン間移動
- Enter/Spaceでのボタンクリック

---

13. 備考

13.1 特記事項

1. **プレゼンテーションコンポーネント**
   - このコンポーネントは表示とイベント発火のみを担当
   - 実際のデータ操作は親コンポーネント（ProgressCheckTable）が担当

2. **selectedMethodIdの未使用**
   - Propsで受け取っているが、現在は使用されていない
   - tasksは親コンポーネントでメソッドによるフィルタリング済みのため不要
   - 将来の拡張（メソッド情報の表示など）のために残されている

3. **多言語対応**
   - 問題確認ボタンのみ多言語対応（useLanguageフックを使用）
   - 他のボタンは日本語固定

- 将来的には全てのラベルを多言語対応する予定

4. **プロジェクトステータスによる制御**
   - isEditableStatus(projectStatus)で編集可能な状態かを判定
   - 'waiting_task'、'executing_task'の場合のみ編集可能
   - 'planning'、'waiting_approval'、'completed'の場合は開始・強制終了ボタンが無効

5. **ステータスの内部関数**
   - getStatusConfig(): コンポーネント外のトップレベルに定義。ステータスバッジの色とラベルを返す
   - getStatusLabel(): `filteredTasks` の `useMemo` 内にインラインで定義。ステータスの日本語ラベルを返す（検索用）

13.2 将来の拡張

1. **タスクのバルク操作**
   - 複数タスクの一括選択
   - 一括開始・一括強制終了

2. **タスクの詳細表示**
   - タスク名クリックで詳細情報を表示
   - サンプル番号、備考、実験記録番号など

3. **フィルタの追加**
   - 進捗率範囲での絞り込み
   - サンプル番号での検索

4. **ソートの拡張**
   - 複数カラムでのソート
   - カスタムソート順の保存

5. **エクスポート機能**
   - CSV/Excelエクスポート

- 絞り込み結果のエクスポート

6. **デバウンス処理**
   - 検索入力のデバウンス（300ms）
   - 過剰な再計算を防ぐ

13.3 関連ドキュメント

- ProgressCheckTable UI設計書 - 親コンポーネント
- ProgressCheckTab UI設計書 - タブコンポーネント
- ViewRendererDialog UI設計書 - フロー図ダイアログ
- experimentWorker型定義 - Task型、ProjectStatus型

13.4 注意事項

1. **カラムタイトル「タスクNo」について**
   - ドキュメントでは「タスクNo」として統一
   - カラム幅は95px

2. **tasksのフィルタリング**
   - このコンポーネントに渡されるtasksは、親コンポーネントで既にメソッドによるフィルタリング済み
   - このコンポーネント内では、検索テキストとステータスによる追加フィルタリングのみを実施

3. **onTaskActionのアクション文字列**
   - 'instruction': 指示確認
   - 'start': タスク開始
   - 'issueCheck': 問題確認
   - 'record': 実験記録
   - 'forceTerminate': 強制終了
   - 親コンポーネントでこれらのアクション文字列を処理する必要がある

14. 変更履歴

バージョン	日付	変更内容	担当者
1.0.2	2026-03-11	ソースコードとの差分を修正（readOnly Props追加、TaskStatus型を4種類に修正、Task型に不足フィールド追加、ページネーションposition削除、問題確認カラム多言語対応明記、getStatusLabel定義場所修正）	Cline
1.0.1	2026-02-23	開始ボタンの説明を更新（楽観的ロック用currentUpdatedAtパラメータ、409 Conflict処理を追加）	Cline
1.0.0	2026-02-20	初版作成	Cline

以上でTaskListTable UI設計書は完成です。